

偏导梯度与链式法则实验

用同一个双参数损失核对五种变化率

先修009与010。实验在CPU上运行，只用本地原创配置和NumPy；不联网、不用GPU、API或自动微分框架。你将先手算，再用独立展开式和有限差分核验，最后故意制造三种错误：漏掉共享变量的路径、把非单位速度当成方向、把存在偏导当成可微。

主计算为 $a=xy$ 、 $z=a+x$ 、 $L=z^2$ 。数学与代码都按第1列 x 、第2列 y 排序。参数点、增量与梯度的shape统一为(1,2)，例如[[2,3]]。不要把(2,)与(2,1)悄悄传给本单元接口。NumPy允许的操作范围比我们的教学接口更广，拒绝某个shape是本实验的明确约定。

A 建立手算基准

在看程序输出前，完成一张纸笔表： $x=2$ 、 $y=3$ 、 $a=6$ 、 $z=8$ 、 $L=64$ ；直接展开的两项偏导为 $2x(y+1)^2$ 和 $2x^2(y+1)$ ，所以梯度是(64,32)。把 x 的贡献再拆成经过 a 的48和直接进入 z 的16。

检查点A：你能说明偏导64与损失64各自表示什么吗？两者这次恰巧都是64，不能只凭同一个数字认为它们是同一种量。梯度有两项，损失只有一项；它们的形状与含义都不同。

读取data/experiment_config.json。point是固定起点；difference_steps是12个正差分步长；linear_scales控制增量($\epsilon, 2\epsilon$)；learning_rates控制四次彼此独立的参数更新。数据都是人为设计的教学数值，没有抽样总体，也不支持模型表现结论。

B 从空状态运行

experiment.py是命令行版本，experiment.ipynb是分步骤的交互版本，二者调用同一套核心函数。test_experiment.py另外用展开多项式的Fraction算术和80位Decimal参照，不把核心gradient函数重新调用一遍当作独立答案。

在011目录、已有Python与NumPy的环境中执行：

```
python experiment.py --output outputs
python test_experiment.py
```

脚本生成outputs/results.json、difference_scan.csv与environment.json。Notebook在本单元目录打开后选择重新启动内核并运行全部，生成notebook_outputs目录下的同名文件。重新执行会覆盖对应输出目录中的这些文件；不会更改data里的配置。要保留自己的对照，请先复制单元，或给脚本传入另一个输出目录。

检查点B: 主结果应为损失64、梯度shape(1,2)和值[[64,32]]; 单元测试应报告passed、9组检查、81个有理数夹具与32个拒绝案例。测试数量还会记录实际执行的文档Python代码块数。程序通过说明这些有限检查通过, 不等于证明所有函数都满足链式法则。

C 写清楚一次局部预测

不用循环更新参数, 只计算从 p 到 $p+\delta$ 的一次变化。下面代码中, `real`是实际损失, `estimate`是全微分给出的线性预测。两个量必须分别命名, 不能将近似结果打印为“真值”。

```
import numpy as np
from experiment import loss, gradient, linear_prediction
p = np.array([[2.0, 3.0]])
d = np.array([[0.01, 0.02]])
real = loss(p + d)
estimate = loss(p) + float((gradient(p) @ d.T)[0, 0])
assert abs(real - 65.28963204) < 1e-12
assert abs(estimate - 65.28) < 1e-12
check = linear_prediction(p, d)
assert abs(check["remainder"] - 0.00963204) < 1e-12
```

再看配置中的 $\epsilon=0.1$ 、 0.01 、 0.001 。每次增量都是 $(\epsilon, 2\epsilon)$, 线性变化应为 128ϵ , 精确余项应为 $96\epsilon^2+32\epsilon^3+4\epsilon^4$ 。计算余项除以 $\sqrt{5}|\epsilon|$, 说明这个比例为何缩小。

检查点C: 写出余项的代数表达式, 再报告有限数值。不要只写“表格显示越来越好, 所以已证明可微”。正文给出了适用于所有足够小增量的界, 才完成本例的数学说明。

D 检查两个坐标偏导

`central_gradient`每次复制起点, 只移动一列, 然后调用完整`loss`重算`a`、`z`、`L`。检查有限差分保持其他独立坐标固定, 不保持由输入计算出来的中间节点固定。

```
from experiment import central_gradient, loss, gradient
p = [[2.0, 3.0]]
numeric = central_gradient(loss, p, 0.01)
np.testing.assert_allclose(numeric, [[64.0, 32.0]], rtol=0, atol=1e-9)
wrong = np.array([[48.0, 32.0]])
assert np.max(np.abs(wrong - gradient(p))) == 16
```

检查点D: 说明错误梯度为什么只在 x 分量上差16。随后用有限差分检查错误梯度, 而不是把`wrong`与另一个由`wrong`计算出来的值比较。一个好的检查器需要独立的信息来源。

观察`difference_scan.csv`。主例对每个单独坐标为二次函数, 所以精确算术中心差分的截断误差为0。结果在浮点中仍可能受表示、相消和舍入影响。不要把它当成“中心差分永远不受步长影响”的证据。

E 区分单位方向和任意速度

先手算 $u=(3/5, 4/5)$ 长度为1，方向导数为64； $v=(3, 4)$ 长度为5，沿 $p+tv$ 的变化率为320。然后运行：

```
from experiment import directional_derivative, rate_along
p = [[2.0, 3.0]]
assert directional_derivative(p, [[0.6, 0.8]]) == 64
assert rate_along(p, [[3.0, 4.0]]) == 320
try:
    directional_derivative(p, [[3.0, 4.0]])
except ValueError as error:
    assert "unit length" in str(error)
else:
    raise AssertionError("Non-unit direction was accepted")
```

检查点E：找一个单位切向方向，使梯度内积为0，并用有限非零增量说明真实损失仍可变化。解释0是“一阶变化率为0”，而不是“无论走多远损失都不变”。归一化必须明确进行；零向量无法变成单位方向。

F 沿联合路线核对链式法则

定义 $x=t$ 、 $y=t+1$ ， $\ell(t)=L(t, t+1)$ 。 $t=2$ 时，两个参数同时以速度1变化。用 $L_x x' + L_y y'$ 得到96，再通过直接展开 $\ell=t^4+4t^3+4t^2$ 独立求导。

```
from experiment import central_curve, rate_along
assert rate_along([[2.0, 3.0]], [[1.0, 1.0]]) == 96
assert abs(central_curve(2.0, 0.1) - 96.12) < 1e-10
assert abs(central_curve(2.0, 0.01) - 96.0012) < 1e-9
```

检查点F：从多项式展开证明中心差分多出的项为 $12h^2$ 。为什么这条路径差分有截断误差，而D任务的坐标差分没有？回答必须提到它们检查的是不同的一元函数。

差分步长 $1e-16$ 在当前起点无法让左右输入都与原点区分，脚本标为Unresolvable input step，并将数值栏留空。空值表示未获得有效结果，不能填0后当成零误差。不同平台最后几位可能变化，不要把某一次最好的 h 写成通用最佳值。

G 让负梯度走得太远

每一行都重新从 $(2, 3)$ 出发，用旧梯度 $(64, 32)$ ，按新点 $= (2, 3) - \eta(64, 32)$ 计算。配置给出的 η 为0.001、0.01、0.1、0.5，并非一个递推序列。

检查点G： $\eta=0.01$ 应给出 $(1.36, 2.68)$ 、损失25.04802304； $\eta=0.5$ 应给出 $(-30, -13)$ 、损失129600。后者反驳“负梯度任意步长都下降”。 $\eta=0.1$ 的一阶预测甚至给负损失，真实值仍是非负的12.3904，这提醒我们不要超出局部近似的适用范围。

用 $q=x^2-y^2$ 在原点的两个轴向取值解释零梯度为什么不能判定最小。不要调用尚未学习的二阶检验，本题只需要直接比较函数值。

H 用反例检查数学条件

固定教学报告的point必须为[[2,3]]，否则run在写文件前拒绝。要探索其他点，请直接调用loss、gradient、linear_prediction等核心函数。

核心函数partials_not_enough实现 $f=xy/\sqrt{x^2+y^2}$ ，原点另外定义为0。名字提醒我们：两个偏导不足以保证全微分存在。它不是主损失，不应把它的不可微性质反推到原来的多项式。

```
import math
from experiment import partials_not_enough, central_gradient
origin = [[0.0, 0.0]]
np.testing.assert_array_equal(
    central_gradient(partials_not_enough, origin, 0.01), [[0.0, 0.0]])
h = 0.01
right = partials_not_enough([[h, h]]) / h
left = partials_not_enough([[-h, -h]]) / (-h)
assert abs(right - 1 / math.sqrt(2)) < 1e-14
assert abs(left + 1 / math.sqrt(2)) < 1e-14
```

检查点H：纸笔证明原点连续、两个偏导均0，并计算沿 $\delta=(t,t)$ 的 $f(\delta)/\|\delta\|_2=1/2$ 。这里的非零常数直接否定可微余项要求。中心差分只比较对称点，可能把尖点的左右差异抵消。

I 审查失败与可复现边界

逐项运行测试中提供的错误shape、一维数组、列数组、布尔值、复数、文本、NaN、无穷值、非正步长、无法分辨的小步长、非单位方向及数值溢出。它们应明确抛出异常，不静默转成貌似正确的梯度。检查central_gradient调用后原始数组没有被原地修改。

提交四部分：A的手算表与两条x路径；C到G的计算和解释；H的反例证明；从空状态执行后的Notebook与脚本输出。对results.json比较内容，对difference_scan.csv比较数值、状态及空值含义。保存自己的实验环境版本。

随附Notebook已在一个新Python进程的真实IPython InProcessKernel中从空状态顺序执行。构建环境禁止socket，所以该记录没有验证浏览器Jupyter界面与跨进程传输；也没有实际新建conda环境。environment.yml是学习者环境说明，核心运行和测试使用已安装的本地依赖。

这些检查覆盖小规模、无量纲、有限实数例子。它们不评估训练速度、真实任务准确率、大模型行为或通用优化收敛性。数值测试辅助发现实现错误，数学假设负责解释公式何时成立。